



Javascript Reference

This document presents the Odoo Javascript framework. This framework is not a large application in term of lines of code, but it is quite generic, because it is basically a machine to turn a declarative interface description into a live application, able to interact with every model and records in the database. It is even possible to use the web client to modify the interface of the web client.

Overview

The Javascript framework is designed to work with three main use cases:

- the *web client*: this is the private web application, where one can view and edit business data. This is a single page application (the page is never reloaded, only the new data is fetched from the server whenever it is needed)
- the *website*: this is the public part of Odoo. It allows an unidentified user to browse some content, to shop or to perform many actions, as a client. This is a classical website: various routes with controllers and some javascript to make it work.
- the *point of sale*: this is the interface for the point of sale. It is a specialized single page application.

Some javascript code is common to these three use cases, and is bundled together (see below in the assets section). This document will focus mostly on the architecture of the web client.

Web client

Single Page Application

The web client is a single-page application: instead of requesting a full page from the server each time the user performs an action, it only loads what is needed to update the user interface (UI) as a result of that action. While doing this, it also takes care of updating information in the URL, so that, in most cases, refreshing the page or closing the browser and opening it again shows you the same thing.

Overview of web client JS code

Here, we give a very quick overview of the web client code, in the *web* addon. The paths will be described relative to *web/static/src*. The following description is deliberately not

- `core/` : this folder contains code that forms the lowest level of the javascript framework and that can be used in the web client as well as the website, portal, and point of sale application.
- `webclient/` : this folder contains files that are specific to the web client and cannot be used in the website or point of sale, such as the action manager and the action service.
- `webclient/webclient.js` : this is the webclient component proper. It is mostly a wrapper for the action container and the navbar, and does a few things that are required upon starting the application, such as loading the state of the url.
- `webclient/actions/` : this folder contains the code responsible for displaying and switching between actions.
- `views/` : this folder contains the code for the view infrastructure, as well as most of the views (some types of views are added by other addons).
- `views/fields/` : contains the definition of the various field components, as well as some utilities used by multiple fields.
- `search/` all these files define the search view (it is not a view in the point of view of the web client, only from the server point of view)

What to do if a file is not loaded/updated

There are many different reasons why a file may not be properly loaded. Here are a few things you can try to solve the issue:

- Make sure you saved your file; forgetting to do that happens to the best of us.
- Take a look at the console (in the dev tools, usually opened with F12) and check for errors.
- Try adding a `console.log()` at the beginning of your file so you can see if a file has been loaded or not. If it is not loaded, it may not be in the proper assets bundle, or the asset bundle may not be up to date.
- Depending on your settings, the server may not regenerate the assets bundles after a file has been modified; there are a few options to solve this:
 - restarting the server will force it to check if the asset bundle is up to date the next time it is requested
 - in debug mode, there is an option in the debug menu (🔧 button in the navbar) to force the server to regenerate the assets bundle on the fly without restarting.
 - starting the server with the `--dev=xml` option will force the server to check if an asset bundle is up to date every time it is requested. We advise you to use this option when actively developing, but not in production.
- Make sure you refresh your page after changing the code. Odoo currently does not have any hot module reloading mechanism.

Some file may need to use code defined in another file. There are two ways of sharing code between files:

- using the global scope (the *window* object) to read/write references to some objects or functions,
- using a module system that will provide a way for each modules to export or import values, and will make sure that they are loaded in a proper order.

While it's possible to work in the global scope, this has a number of issues:

- It is difficult to ensure that implementation details are not exposed: function declarations in the global scope are accessible to all other code.
- There is a single namespace, creating great potential for naming conflicts.
- Dependencies are implicit: if a piece of code depends on another, the order in which they are loaded is important, but difficult to guarantee.

Using a module system helps resolve these issues: because modules specify their dependencies, the module system can load them in the proper order or emit an error if dependencies are missing or circular. Modules also form their own namespace, and can choose what to export, preventing exposure of implementation detail and naming collisions.

While we could use ECMAScript (ES) modules directly, there are a number of disadvantages to that approach: each ES module requires a network round trip, which becomes very slow when you have hundreds of files, and many files in Odoo need to be present despite not being imported by anything because they simply add code that the framework will use instead of the other way around.

Because of this, Odoo has a system of asset bundles. In these bundles, JavaScript files are ES modules with a special annotation at the top. These modules will be bundled together and transpiled to be usable by our module loader. While you can write code that doesn't use this module system, it is generally not recommended.

(see [Native Javascript Modules](#))

Patching classes

While we do our best to provide extension points that don't require it, it is sometimes necessary to modify the behavior of an existing class *in place*. The goal is to have a mechanism to change a class and all future/present instances. This is done by using the `patch` utility function:

```
patch(Hamster.prototype, {
    sleep() {
        super.sleep(...arguments);
        console.log("zzzz");
    },
});
```

When patching methods, you need to patch the class' prototype, but if you would like to patch a static property of the class, you need to patch the class itself.

Patching is a dangerous operation and should be done with care as it will modify all instances of the class, even if they have already been created. To avoid weird issues, patches should be applied as soon as possible, at the top-level of your module. Patching classes at runtime can result in extremely difficult to debug issues if the class has already been instantiated.

Registries

A common need in the Odoo ecosystem is to extend/change the behaviour of the base system from the outside (by installing an application, i.e. a different module). For example, one may need to add a new field widget in some views. In that case, and many others, the usual process is to create the desired component, then add it to a registry (registering step), to make the rest of the web client aware of its existence.

There are a few registries available in the system. The registries that are used by the framework are categories on the main registry, that can be imported from

@web/core/registry

field registry

The field registry contains all field widgets known to the web client. Whenever a view (typically form or list/kanban) needs a field widget, this is where it will look. A typical use case look like this:

```
import { registry } from "@web/core/registry";
class PadField extends Component { ... }

registry.category("fields").add("pad", {
    component: PadField,
    supportedTypes: ["char"],
```

view registry

This registry contains all JS views known to the web client.

action registry

We keep track of all client actions in this registry. This is where the action manager looks up whenever it needs to create a client action. Client actions can be a function - the function will be called when the action is invoked, and the returned value will be executed as a follow up action if needed - or an Owl component that will be displayed when executing that action.

Services

Within the webclient, there are some concerns that cannot be handled by a single component, as the concern is transversal, involves many components, or needs to maintain some state for as long as the application is alive.

Services are a solution to these problems: they are created during application startup, are available to components through the hook `useService`, and stay alive for the entire lifetime of the application.

For example, we have the *orm* service whose job is to allow interacting with business objects on the server.

Here is a simplified example on how the *orm* service is implemented:

```
import { registry } from "@web/core/registry";
export const OrmService = {
  start() {
    return {
      read(...) { ... },
      write(...) { ... },
      unlink(...) { ... },
      ...
    }
  },
};
registry.category("services").add("orm", OrmService);
```

Using services

destroyed during the call.

```
class SomeComponent extends Component {
  setup() {
    this.orm = useService("orm");
  }
  // ...
  getActivityModelViewID(model) {
    return this.orm.call(model, "get_activity_view_id", this.params);
  }
}
```

Talking to the server

There are typically two use cases when working on Odoo: one may need to call a method on a (python) model (this goes through the controller `/web/dataset/call_kw`), or one may need to directly call a controller (available on some route).

- Calling a method on a python model is done through the orm service:

```
return this.orm.call("some.model", "some_method", [some, args]);
```

- Directly calling a controller is done through the rpc service:

```
return this.rpc("/some/route/", {
  some: param,
});
```

Note

The rpc service doesn't really perform what is generally understood as a remote procedure call (RPC), but for historical reasons, within Odoo we generally call any network request performed in JavaScript an RPC. As highlighted in the previous paragraph, if you want to call a method on a model, you should use the orm service.

Notifications

- *info*: useful to display some informational feedback as a consequence of an action that cannot fail.
- *success*: the user performed an action that can sometimes fail but didn't.
- *warning*: the user performed an action that could only be partially completed. Also useful if something is wrong but wasn't caused by the user directly, or is not particularly actionable.
- *success*: the user tried to performed an action but it couldn't be completed.

Notifications can also be used to ask a question to the user without disturbing their workflow: e.g. a phone call received through VOIP: a sticky notification could be displayed with two buttons to *Accept* or *Decline*.

Displaying notifications

There are two ways to display notifications in Odoo:

- The *notification* service allows component to display notifications from JS code by calling the add method.
- The *display_notification* client action allows to trigger the display of a notification from python (e.g. in the method called when the user clicked on a button of type object). This client action uses the notification service.

Notifications have a few *options*:

- *title*: string, optional. This will be displayed on the top as a title.
- *message*: string, optional. The content of the notification. Can be a markup object to display formatted text.
- *sticky*: boolean, optional (default false). If true, the notification will stay until the user dismisses it. Otherwise, the notification will be automatically closed after a short delay.
- *type*: string, optional (default "warning"). Determines the style of the notification. Possible values: "info", "success", "warning", "danger"
- *className*: string, optional. This is a css class name that will be automatically added to the notification. This could be useful for styling purpose, even though its use is discouraged.

Here are some examples on how to display notifications in JS:

```
// note that we call _t on the text to make sure it is properly translated.  
this.notification.add({
```



```
title: _t("Error"),
message: _t("Filter name is required."),
type: "danger",
});
```

And in Python:

```
# note that we call _(string) on the text to make sure it is properly translated.
def show_notification(self):
    return {
        'type': 'ir.actions.client',
        'tag': 'display_notification',
        'params': {
            'title': _('Success'),
            'message': _('Your signature request has been sent.'),
            'sticky': False,
        }
    }
```

Systray

The Systray is the right part of the navbar in the interface, where the web client displays a few widgets, such as the messaging menu.

When the systray is created by the navbar, it will look for all registered systray items and display them.

There is currently no specific API for systray items. They are Owl components, and can communicate with their environment just like other components, e.g. by interacting with services.

Adding a new Systray Item

Items can be added to the systray by adding them to the “systray” registry:

```
import { registry } from "@web/core/registry"
class MySystrayComponent extends Component {
    ...
```


The items are ordered in the systray according to their sequence in the systray registry.

Translation management

Some translations are made on the server side (basically all text strings rendered or processed by the server), but there are strings in the static files that need to be translated. The way it currently works is the following:

- each translatable string is tagged with the special function `_t`
- these strings are used by the server to generate the proper PO files
- whenever the web client is loaded, it will call the route `/web/webclient/translations`, which returns a list of all translatable terms
- at runtime, whenever the function `_t` is called, it will look up in this list in order to find a translation, and return it or the original string if none is found.

Note that translations are explained in more details, from the server point of view, in the document [Translating Modules](#).

```
import { _t } from "@web/core/l10n/translation";

class SomeComponent extends Component {
  static exampleString = _t("this should be translated");
  ...
  someMethod() {
    const str = _t("some text");
  }
}
```

Note that using the translation function requires some care: the string given as an argument cannot be dynamic, as it is extracted statically from the code to generate the PO files and serves as the identifier for the term to translate. If you need to inject some dynamic content in the string, `_t` supports placeholders:

```
import { _t } from "@web/core/l10n/translation";
const str = _t("Hello %s, you have %s unread messages.", user.name, unreadCount);
```

Session

The webclient needs some information from the python to function properly. To avoid an extra round-trip with the server by making a network request in JavaScript, this information is serialized directly in the page, and can be accessed in JS through the `@web/session` module.

Adding information to the session

When the `/web` route is loaded, the server injects this information in a script tag. The information is obtained by calling the method `session_info` of the model `ir.http`. You can override this method to add information to the returned dictionary.

```
from odoo import models
from odoo.http import request

class IrHttp(models.AbstractModel):
    _inherit = ['ir.http']

    def session_info(self):
        result = super(IrHttp, self).session_info()
        result['some_key'] = get_some_value_from_db()
        return result
```

Now, the value can be obtained in javascript by reading it in the session:

```
import { session } from "@web/session"
const myValue = session.some_key;
...
```

Note that this mechanism is designed to reduce the amount of communication needed by the web client to be ready. It is only appropriate for data which is cheap to compute (a slow `session_info` call will delay the loading for the web client for everyone), and for data which is required early in the initialization process.

Views

component called "SomethingController" which is the root of the view. This component creates an instance of some "model" (an object responsible for managing the data), and has a subcomponent called a "renderer" that handles the display logic.

Fields

A good part of the web client experience is about editing and creating data. Most of that work is done with the help of field widgets, which are aware of the field type and of the specific details on how a value should be displayed and edited.

Decorations

Like the list view, field widgets have a simple support for decorations. The goal of decorations is to have a simple way to specify a text color depending on the record current state. For example:

```
<field name="state" decoration-danger="amount < 10000"/>
```

The valid decoration names are:

- decoration-bf
- decoration-it
- decoration-danger
- decoration-info
- decoration-muted
- decoration-primary
- decoration-success
- decoration-warning

Each decoration *decoration-X* will be mapped to a css class *text-X*, which is a standard bootstrap css class (except for *text-it* and *text-bf*, which are handled by odoo and correspond to italic and bold, respectively). Note that the value of the decoration attribute should be a valid python expression, which will be evaluated with the record as evaluation context.

Non-relational fields

We document here all non relational fields available by default, in no particular order.

Options:

- `type` : setting the input type ("text" by default, can be set on "number")

In edit mode, the field is rendered as an input with the HTML attribute type set on "number" (so user can benefit the native support, especially on mobile). In this case, the default formatting is disabled to avoid incompatibility.

```
<field name="int_value" options="{ 'type': 'number' }" />
```

- `step` : set the step to the value up and down when the user click on buttons (only for input of type number, 1 by default)

```
<field name="int_value" options="{ 'type': 'number', 'step': 100 }" />
```

- `format` : should the number be formatted. (true by default)

By default, numbers are formatted according to locale parameters. This option will prevent the field's value from being formatted.

```
<field name="int_value" options='{ "format": false }' />
```

Float (float)

This is the default field type for fields of type float .

- Supported field types: float

Attributes:

- `digits` : displayed precision

```
<field name="factor" digits="[42,5]" />
```

Options:

the default formatting is disabled to avoid incompatibility.

```
<field name="float_value" options="{ 'type': 'number' }" />
```

- `step` : set the step to the value up and down when the user click on buttons (only for input of type number, 1 by default)

```
<field name="float_value" options="{ 'type': 'number', 'step': 0.1 }" />
```

- `format` : should the number be formatted. (`true` by default)

By default, numbers are formatted according to locale parameters. This option will prevent the field's value from being formatted.

```
<field name="float_value" options="{ 'format': false }" />
```

- `min_display_digits` : The minimum number of decimal digits to display.

For example, if set to 3 and no decimal precision is provided: 1.2 becomes "1.200", 1.123 becomes "1.123" and 1.1234 becomes "1.1234".

```
<field name="float_value" options="{ 'min_display_digits': 3 }" />
```

- `hide_trailing_zeros` : hide zeros to the right of the last non-zero digit, e.g. 1.20 becomes 1.2 (`false` by default).

```
<field name="float_value" options="{ 'hide_trailing_zeros': true }" />
```

Time (`float_time`)

The goal of this widget is to display properly a float value that represents a time interval (in hours). So, for example, 0.5 should be formatted as 0:30, or 4.75 correspond to 4:45.

- Supported field types: `float`

value should be formatted as 1.5 .

- Supported field types: float

Float Toggle (float_toggle)

The goal of this widget is to replace the input field by a button containing a range of possible values (given in the options). Each click allows the user to loop in the range. The purpose here is to restrict the field value to a predefined selection. Also, the widget support the factor conversion as the float_factor widget (Range values should be the result of the conversion).

- Supported field types: float

```
<field name="days_to_close" widget="float_toggle" options="{ 'factor': 2, 'range':
```

Boolean (boolean)

This is the default field type for fields of type boolean .

- Supported field types: boolean

Char (char)

This is the default field type for fields of type char .

- Supported field types: char

Date (date)

This is the default field type for fields of type date . It consists of a text box and a date picker. The field will show the date in a readable format like this: Dec 19, 1997 . The year will be hidden if it's the current one. When editing the field, the numeric format will be shown. This format corresponds to the one set in the current language.

- Supported field types: date

Options:

- min_date / max_date : sets limit dates for accepted values. By default, the earliest accepted date is **1000-01-01** and the latest is **9999-12-31**. Accepted values are SQL-formatted dates (yyyy-MM-dd HH:mm:ss) or "today" .

- `warn_future` : displays a warning if the value is in the future (based on today).

```
<field name="datefield" options="{ 'warn_future': true }" />
```

- `numeric` : when set to true, it shows the date in the format set on the current language. (default: false).

```
<field name="datefield" options="{ 'numeric': true }" />
```

Date & Time (`datetime`)

This is the default field type for fields of type `datetime` . The values are always in the client's timezone. The displayed format has the same behaviour as the date field, see [Date Field](#) description. The readable format looks like this: Dec 19, 1997, 10:45 AM .

- Supported field types: `datetime`

Options:

- see [Date Field](#) options
- `rounding` : increment used to generate available minutes in the time picker. This does not affect the actual value, just the amount of available options in the select dropdown (default: 5).

```
<field name="datetimefield" options="{ 'rounding': 10 }" />
```

- `show_seconds` : when set to true, it shows the seconds from the datetime field. The field will still accept datetime values, but the seconds will be shown in the UI (default: false).

```
<field name="datetimefield" widget="datetime" options="{ 'show_seconds': true }" />
```

- `show_time` : when set to false, it hides the time part from the datetime field. The field will still accept datetime values, but the time part will be hidden in the UI (default: true).

- `show_date` : when set to false, it hides the date part from the datetime field. The field will still accept datetime values, but the date part will be hidden in the UI (default: `true`).

```
<field name="datetimefield" widget="datetime" options="{ 'show_date': false }" />
```

Date Range (`daterange`)

This widget allows the user to select start and end date from a single picker.

- Supported field types: `date` , `datetime`

Options:

- see [Date Field](#) or [Date & Time Field](#) options
- `start_date_field` : field used to get/set the start value of the date range (cannot be used with `end_date_field`).

```
<field name="end_date" widget="daterange" options="{ 'start_date_field': 'start_date' }" />
```

- `end_date_field` : field used to get/set the end value of the date range (cannot be used with `start_date_field`).

```
<field name="start_date" widget="daterange" options="{ 'end_date_field': 'end_date' }" />
```

Remaining Days (`remaining_days`)

This widget can be used on date and datetime fields. In readonly, it displays the delta (in days) between the value of the field and today. The widget turns into a regular date or datetime field in edit mode.

- Supported field types: `date` , `datetime`

Monetary (`monetary`)

This is the default field type for fields of type `monetary` . It is used to display a currency. If there is a currency fields given in option, it will use that, otherwise it will fall back to the

Options:

- `currency_field` : another field name which should be a many2one on currency.

```
<field name="value" widget="monetary" options="{ 'currency_field': 'currency_id' }" />
```

- `hide_trailing_zeros` : hide zeros to the right of the last non-zero digit, e.g. 1.20 becomes 1.2 (false by default).

```
<field name="int_value" options="{ 'hide_trailing_zeros': true }" />
```

Text (`text`)

This is the default field type for fields of type `text` .

- Supported field types: `text`

Handle (`handle`)

This field's job is to be displayed as a `handle` , and allows reordering the various records by drag and dropping them.

Warning

It has to be specified on the field by which records are sorted.

Warning

Having more than one field with a `handle` widget on the same list is not supported.

- Supported field types: `integer`

Email (`email`)

This field displays email address. The main reason to use it is that it is rendered as an anchor tag with the proper href, in readonly mode.

- Supported field types: `char`

Phone (`phone`)

- Supported field types: `char`

URL (`url`)

This field displays an url (in readonly mode). The main reason to use it is that it is rendered as an anchor tag with the proper css classes and href.

Also, the text of the anchor tag can be customized with the `text` attribute (it won't change the href value).

- Supported field types: `char`

```
<field name="foo" widget="url" text="Some URL" />
```

Options:

- `website_path` : (default: `false`) by default, the widget forces (if not already the case) the href value to begin with `"http://"` except if this option is set to `true` , thus allowing redirections to the database's own website.

Domain (`domain`)

The `domain` field allows the user to construct a technical-prefix domain thanks to a tree-like interface and see the selected records in real time. In debug mode, an input is also there to be able to enter the prefix char domain directly (or to build advanced domains the tree-like interface does not allow to).

Note that this is limited to **static** domains (no dynamic expressions, or access to context variable).

- Supported field types: `char`

Options:

- `model` : the name of the char field encoding the `res_model` on which the domain applies.
- `foldable` (default: `false`): if true, the domain field is rendered compactly and unfolds itself upon user interaction.
- `in_dialog` (default: `false`): if true, the widget opens a dialog when the user wants to edit the domain whereas by default, the domain editor is rendered just below the value.
- `count_limit` (default: `10000`): the domain widget performs `search_count` requests to validate the domain, and to indicate the number of records matching it. This requests

content. The link is clickable and will open a new browser window with its value as url.

- Supported field types: char

Image File (`image`)

This widget is used to represent a binary value as an image. In some cases, the server returns a `bin_size` instead of the real image (a `bin_size` is a string representing a file size, such as "6.5kb"). In that case, the widget will make an image with a source attribute corresponding to an image on the server.

- Supported field types: binary

Options:

- `preview_image` : if the image is only loaded as a `bin_size` , then this option is useful to inform the web client that the default field name is not the name of the current field, but the name of another field.

```
<field name="image" widget="image" options="{ 'preview_image': 'image_128' }" />
```

- `accepted_file_extensions` : the file extension the user can pick from the file input dialog box (default value is "image/*")

(cf: `accept` attribute on `<input type="file" />`)

Binary File (`binary`)

Generic widget to allow saving/downloading a binary file.

- Supported field types: binary

Attributes:

- `filename` : saving a binary file will lose its file name, since it only saves the binary value. The file name can be saved in another field. To do that, a `filename` attribute should be set to a field present in the view.

```
<field name="datas" filename="datas_fname" />
```

(cf: `accept` attribute on `<input type="file" />`)

Priority (`priority`)

This widget is rendered as a set of stars, allowing the user to click on it to select a value or not. This is useful for example to mark a task as high priority.

Note that this widget also works in `readonly` mode, which is unusual.

- Supported field types: `selection`

Image Attachment (`attachment_image`)

Image widget for `many2one` fields. If the field is set, this widget will be rendered as an image with the proper `src` url. This widget does not have a different behaviour in edit or `readonly` mode, it is only useful to view an image.

- Supported field types: `many2one`

```
<field name="displayed_image_id" widget="attachment_image" />
```

Label Selection (`label_selection`)

This widget renders a simple non-editable label. This is only useful to display some information, not to edit it.

- Supported field types: `selection`

Options:

- `classes` : a mapping from a selection value to a CSS class name

```
<field
  name="state"
  widget="label_selection"
  options="{
    'classes': {
      'draft': 'default',
      'cancel': 'default',
      'none': 'danger',
    },
  },
  />
```

State Selection (`state_selection`)

This is a specialized selection widget. It assumes that the record has some hardcoded fields, present in the view: `stage_id` , `legend_normal` , `legend_blocked` , `legend_done` . This is mostly used to display and change the state of a task in a project, with additional information displayed in the dropdown.

- Supported field types: `selection`

```
<field name="kanban_state" widget="state_selection" />
```

State Selection - List View (`list.state_selection`)

In list views, the `state_selection` field displays by default the label next to the icon.

- Supported field types: `selection`

Options:

- `hide_label` : hide the label next to the icon

```
<field name="kanban_state" widget="state_selection" options="{ 'hide_label': true }" />
```

Favorite (`boolean_favorite`)

This widget is displayed as an empty (or not) star, depending on a boolean value. Note that it also can be edited in readonly mode.

- Supported field types: `boolean`

Toggle (`boolean_toggle`)

Displays a toggle switch to represent a boolean. This is a subfield of the `boolean` field, mostly used to have a different look.

- Supported field types: `boolean`

Stat Info (`statinfo`)

This widget is meant to represent statistical information in a `stat` button . It is basically just a label with a number.

- Supported field types: `integer` , `float`

```

<button
  name="%(act_payslip_lines)d"
  icon="fa-money"
  type="action"
>
  <field
    name="payslip_count"
    widget="statinfo"
    string="Payslip"
    options="{ 'label_field': 'label_tasks' }"
  />
</button>

```

Percent Pie (`percentpie`)

This widget is meant to represent statistical information in a `stat` button . This is similar to a `statinfo` widget, but the information is represented in a **pie chart** (empty to full). Note that the value is interpreted as a percentage (a number between 0 and 100).

- Supported field types: `integer` , `float`

```
<field name="replied_ratio" string="Replied" widget="percentpie" />
```

Progress Bar (`progressbar`)

Represent a value as a progress bar (from 0 to some value)

- Supported field types: `integer` , `float`

Options:

- `editable` : boolean determining whether the `value` is editable
- `current_value` : get the current value from the field that must be present in the view
- `max_value` : get the maximum value from the field that must be present in the view
- `edit_max_value` : boolean determining whether the `max_value` is editable
- `title` : title of the bar, displayed on top of the bar

→ not translated, use `title` attribute (not option) instead if the term must be translated

```
options="{
    'current_value': 'absence_of_today',
    'max_value': 'total_employee',
    'editable': false,
}"
/>
```

Journal Dashboard Graph (`dashboard_graph`)

This is a more specialized widget, useful to display a graph representing a set of data. For example, it is used in the accounting dashboard kanban view.

It assumes that the field is a JSON serialization of a set of data.

- Supported field types: `char`

Attributes:

- `graph_type` : string, can be either `"line"` or `"bar"`

```
<field name="dashboard_graph_data" widget="dashboard_graph" graph_type="line"
```

Ace Editor (`ace`)

This widget is intended to be used on Text fields. It provides Ace Editor for editing XML and Python.

- Supported field types: `char` , `text`

Badge (`badge`)

Displays the value inside a bootstrap badge pill.

- Supported field types: `char` , `selection` , `many2one`

By default, the badge has a light grey background, but it can be customized by using the **Decoration** mechanism. For instance, to display a red badge under a given condition:

```
<field name="foo" widget="badge" decoration-danger="state == 'cancel'" />
```

Relational fields

Attributes:

- `placeholder` : a string which is used to display some info when no value is selected

```
<field name="tax_id" widget="selection" placeholder="Select a tax" />
```

Radio (`radio`)

This is a subfield of `FieldSelection`, but specialized to display all the valid choices as radio buttons.

Note that if used on a many2one records, then more rpcs will be done to fetch the `name_gets` of the related records.

- Supported field types: `selection`, `many2one`

Options:

- `horizontal` : if `true`, radio buttons will be displayed horizontally.

```
<field name="recommended_activity_type_id" widget="radio" options="{ 'horizontal": true }" />
```

Badge Selection (`selection_badge`)

This is a subfield of `selection` field, but specialized to display all the valid choices as rectangular badges.

- Supported field types: `selection`, `many2one`

```
<field name="recommended_activity_type_id" widget="selection_badge" />
```

Many2one (`many2one`)

Default widget for many2one fields.

- Supported field types: `many2one`

Attributes:

Options:

- `quick_create` : allow the quick creation of related records (default: `true`)
- `no_create` : prevent the creation of related records - hide both the **Create "xxx"** and **Create and Edit** dropdown menu items (default: `false`)
- `no_quick_create` : prevent the quick creation of related records - hide the **Create "xxx"** dropdown menu item (default: `false`)
- `no_create_edit` : hide the **Create and Edit** dropdown menu item (default: `false`)
- `create_name_field` : when creating a related record, if this option is set, the value of the `create_name_field` will be filled with the value of the input (default: `name`)
- `always_reload` : boolean, default to `false` . If `true` , the widget will always do an additional `name_get` to fetch its name value. This is used for the situations where the `name_get` method is overridden (please do not do that)
- `no_open` : boolean, default to `false` . If set to `true` , the many2one will not redirect on the record when clicking on it (in readonly mode)

```
<field name="currency_id" options="{ 'no_create': true, 'no_open': true }" />
```

Many2one Barcode (`many2one_barcode`)

Widget for `many2one` fields allows to open the camera from a mobile device (Android/iOS) to scan a barcode.

Specialization of `many2one` field where the user is allowed to use the native camera to scan a barcode. Then it uses `name_search` to search this value.

If this widget is set and user is not using the mobile application, it will fallback to regular `many2one` (`Many2oneField`)

- Supported field types: `many2one`

Many2one Avatar (`many2one_avatar`)

This widget is only supported on `many2one` fields pointing to a model which inherits from `image.mixin` . In readonly, it displays the image of the related record next to its `display_name` . Note that the `display_name` isn't a clickable link in this case. In edit, it behaves exactly like the regular `many2one` .

- Supported field types: `many2one`

Many2one Avatar User (`many2one_avatar_user`)

- Supported field types: `many2one` (pointing to `res.users`)

Many2one Avatar Employee (`many2one_avatar_employee`)

Same as `many2one_avatar_user` , but for `many2one` fields pointing to `hr.employee` .

- Supported field types: `many2one` (pointing to `hr.employee`)

Many2many (`many2many`)

Default widget for `many2many` fields.

- Supported field types: `many2many`

Attributes:

- `mode` : string, default view to display
- `domain` : restrict the data to a specific domain

Options:

- `create_text` : allow the customization of the text displayed when adding a new record
- `link` : domain determining whether records can be added to the relation (default: `true`).
- `unlink` : domain determining whether records can be removed from the relation (default: `true`).

Many2many Binary File (`many2many_binary`)

This widget helps the user to upload or delete one or more files at the same time.

Note that this widget is specific to the model `ir.attachment` .

- Supported field types: `many2many`

Options:

- `accepted_file_extensions` : the file extension the user can pick from the file input dialog box

(cf: `accept` attribute on `<input type="file" />`)

Many2many Tags (`many2many_tags`)

Display a `many2many` field as a list of tags.

- Supported field types: `many2many`

```
<field name="category_id" widget="many2many_tags" options="{ 'create': [['some_
```

- `color_field` : the name of a numeric field, which should be present in the view. A color will be chosen depending on its value.

```
<field name="category_id" widget="many2many_tags" options="{ 'color_field': 'co
```

- `no_edit_color` : set to `true` to remove the possibility to change the color of the tags (default: `false`).

```
<field name="category_id" widget="many2many_tags" options="{ 'color_field': 'co
```

- `edit_tags` : set to `true` to add the possibility to update tag related record by clicking on the tags. (default: `false`).

```
<field name="category_id" widget="many2many_tags" options="{ 'edit_tags': true]
```

Many2many Tags - Form View (`form.many2many_tags`)

Specialization of `many2many_tags` widget for form views. It has some extra code to allow editing the color of a tag.

- Supported field types: `many2many`

Many2many Tags - Kanban View (`kanban.many2many_tags`)

Specialization of `many2many_tags` widget for kanban views.

- Supported field types: `many2many`

Many2many Checkboxes (`many2many_checkboxes`)

This field displays a list of checkboxes and allows the user to select a subset of the choices. Note that the number of displayed values is limited to `100` . This limit isn't customizable. It simply allows to handle extreme cases where this widget is wrongly set on a field with a

One2many (one2many)

Default widget for `one2many` fields. It usually displays data in a sub list view, or a sub kanban view.

- Supported field types: `one2many`

Options:

- `create` : domain determining whether or not related records can be created (default: `true`).
- `delete` : domain determining whether or not related records can be deleted (default: `true`).

```
<field name="turtles" options="{ 'create': [['some_other_field', '>', 24]]}" />
```

- `create_text` : a string that is used to customize the 'Add' label/text.

```
<field name="turtles" options="{ 'create_text': 'Add turtle'}" />
```

Status Bar (statusbar)

This is a field specific to the form views. It is the bar on top of many forms which represent a flow, and allow selecting a specific state.

- Supported field types: `selection` , `many2one`

Reference (reference)

The `reference` field is a combination of a `select` (for the model) and a `many2one` field (for its value). It allows the selection of a record on an arbitrary model.

- Supported field types: `char` , `reference`

Options:

- `model_field` : name of an `ir.model` containing the model of the records that can be selected. When this option is set, the `select` part of the `reference` field isn't displayed.

sheet, for instance, to indicate archived records.

```
<widget name="web_ribbon" title="Archived" bg_color="text-bg-danger"/>
```

Attributes:

- `title` : text displayed in the ribbon.
- `tooltip` : text displayed in the ribbon's tooltip.
- `bg-class` : classname to set on the ribbon, typically to define the ribbon's color.

Week Days (`week_days`)

This widget displays a list of checkboxes for week days, 1 checkbox for each day and allow the user to select a subset of the choices.

```
<widget name="week_days" />
```

Client actions

A client action is a component that can be displayed as the main element in the webclient, occupying all the space below the navbar, just like an `act_window_action`. This is useful when you need a component that is not closely linked to an existing view or a specific model. For example, the Discuss application is a client action.

A client action is a term that has various meanings, depending on the context:

- from the perspective of the server, it is a record of the model `ir_action`, with a field `tag` of type char
- from the perspective of the web client, it is an Owl component registered in the action registry under the same key its tag

Whenever a menu item is associated with a client action, opening it will simply fetch the action definition from the server, then lookup its tag in the action registry to get the component definition. This component will then be rendered by the action container.

Adding a client action

A client action is a component that will control the part of the screen below the navbar.

Defining a client action is as simple as creating an Owl component and adding it to the action

```
class MyClientAction extends Component { ... }  
registry.category("actions").add("my-custom-action", ClientAction);
```

Then, to use the client action in the web client, we need to create a client action record (a record of the model `ir.actions.client`) with the proper `tag` attribute:

```
<record id="my_client_action" model="ir.actions.client">  
  <field name="name">Some Name</field>  
  <field name="tag">my-custom-action</field>  
</record>
```

Get Help

[Contact Support](#)[Ask the Odoo Community](#)